```
"""
engine.py
---------
Main entry point. Ties fundamental.py (currency strength) and
technical.py (price structure) together into a single cross-referenced
view per pair.

Usage:
    python engine.py --fundamentals fundamental_data_template.csv --pairs EURUSD,
    python engine.py --demo          # runs fully offline on synthetic data

For each pair BASE/QUOTE, it computes:
  FundamentalBias = strength(BASE) - strength(QUOTE)   (which side fundamentals f
  TechnicalScore  = from technical.py                  (which side price structur
  Agreement       = do the signs match?
  ConvictionScore = |FundamentalBias(normalized)| * |TechnicalScore| if they agre

Pairs are ranked by ConvictionScore descending -- the top of the list is
where fundamentals and technicals are pulling the same direction with the
most conviction on both sides, i.e. the "clearest market structure."
"""

import argparse
import os
import sys
import pandas as pd
import numpy as np

from fundamental import strength_table, load_fundamental_data, compute_currency_s
from technical import compute_technical_signals


def default_pairs(currencies):
    """Builds a reasonable default set of pairs: everything against USD,
    plus a few common crosses, restricted to currencies actually present
    in the fundamental data. Follows standard FX market convention for
    which currency is quoted as the base (e.g. EURUSD, but USDJPY/USDCAD/USDCHF).
    pairs = []
    usd_is_base = ["JPY", "CAD", "CHF"]   # quoted as USD<X>
    usd_is_quote = ["EUR", "GBP", "AUD", "NZD"]  # quoted as <X>USD
    for c in usd_is_base:
        if c in currencies and "USD" in currencies:
            pairs.append(f"USD{c}")
    for c in usd_is_quote:
```

```
        if c in currencies and "USD" in currencies:
            pairs.append(f"{c}USD")
    crosses = ["EURGBP", "EURJPY", "GBPJPY", "AUDJPY", "EURCHF"]
    for p in crosses:
        base, quote = p[:3], p[3:]
        if base in currencies and quote in currencies:
            pairs.append(p)
    return pairs


def cross_reference(fundamentals_csv: str, pairs: list, use_synthetic: bool = Fal
                    period: str = "6mo", interval: str = "1d") -> pd.DataFrame:
    df_raw = load_fundamental_data(fundamentals_csv)
    strength = compute_currency_strength(df_raw)
    # normalize composite score to roughly [-1, 1] for comparability with Technic
    max_abs = strength["CompositeScore"].abs().max() or 1.0
    norm_strength = strength["CompositeScore"] / max_abs

    rows = []
    for pair in pairs:
        base, quote = pair[:3], pair[3:]
        if base not in norm_strength.index or quote not in norm_strength.index:
            print(f"[skip] {pair}: missing fundamental data for {base} or {quote}
            continue

        fundamental_bias = norm_strength[base] - norm_strength[quote]

        if use_synthetic:
            from data_feed import generate_synthetic_ohlc
            seed = abs(hash(pair)) % (2**32)
            ohlc = generate_synthetic_ohlc(n=220, seed=seed)
        else:
            from data_feed import get_ohlc
            ohlc = get_ohlc(pair, period=period, interval=interval)

        tech = compute_technical_signals(ohlc)
        technical_score = tech["TechnicalScore"]

        agree = np.sign(fundamental_bias) == np.sign(technical_score) and fundame
        conviction = abs(fundamental_bias) * abs(technical_score)
        if not agree:
            conviction *= 0.3  # penalize disagreement heavily rather than zeroin

        rows.append({
            "Pair": pair,
            "Base": base, "Quote": quote,
            "FundamentalBias": round(float(fundamental_bias), 3),
```

```
            "TechnicalScore": technical_score,
            "Structure": tech["Structure"],
            "Agreement": bool(agree),
            "ConvictionScore": round(float(conviction), 3),
            "RSI14": tech["RSI14"],
        })

    result = pd.DataFrame(rows).sort_values("ConvictionScore", ascending=False).r
    return result


def main():
    parser = argparse.ArgumentParser(description="Forex fundamental x technical c
    parser.add_argument("--fundamentals", default="fundamental_data_template.csv"
                        help="Path to your filled-in fundamental data CSV")
    parser.add_argument("--pairs", default=None,
                        help="Comma-separated pairs e.g. EURUSD,GBPUSD,USDJPY (d
    parser.add_argument("--period", default="6mo", help="yfinance lookback period
    parser.add_argument("--interval", default="1d", help="yfinance interval (e.g.
    parser.add_argument("--demo", action="store_true",
                        help="Run fully offline using synthetic price data (no n
    parser.add_argument("--fetch-fundamentals", action="store_true",
                        help="Auto-fetch fundamental data from FRED instead of r
                             "Requires --fred-key or FRED_API_KEY env var.")
    parser.add_argument("--fred-key", default=os.environ.get("FRED_API_KEY"),
                        help="FRED API key (get one free at https://fredapi.stlo
    parser.add_argument("--currencies", default="USD,EUR,GBP,JPY,AUD,CAD,CHF,NZD,
                        help="Comma-separated currencies to fetch fundamentals f
    args = parser.parse_args()

    if args.fetch_fundamentals:
        if not args.fred_key:
            print("Error: --fetch-fundamentals requires --fred-key or FRED_API_KE
                  "Get a free key at https://fredapi.stlouisfed.org/docs/api/api_
            sys.exit(1)
        from fred_fetch import build_fundamental_dataframe
        currencies = [c.strip().upper() for c in args.currencies.split(",")]
        print(f"Fetching fundamental data from FRED for {len(currencies)} currenc
        fetched = build_fundamental_dataframe(args.fred_key, currencies)
        fetched.to_csv(args.fundamentals)
        print(f"Auto-fetched fundamentals saved to {args.fundamentals} "
              f"(open it to review/edit before trusting the results)\n", file=sys

    df_raw = load_fundamental_data(args.fundamentals)
    currencies = list(df_raw.index)

    print("=== 1. Fundamental Currency Strength Ranking ===")
```

```
    table = strength_table(args.fundamentals)
    print(table.to_string(float_format=lambda x: f"{x:.2f}"))
    print(f"\nStrongest: {table.index[0]}   |   Weakest: {table.index[-1]}\n")

    pairs = args.pairs.split(",") if args.pairs else default_pairs(currencies)
    pairs = [p.strip().upper() for p in pairs]

    print(f"=== 2. Technical + Cross-Reference across {len(pairs)} pairs ===")
    if args.demo:
        print("(using synthetic price data -- offline demo mode)\n")
    result = cross_reference(args.fundamentals, pairs, use_synthetic=args.demo,
                             period=args.period, interval=args.interval)
    if result.empty:
        print("No pairs could be evaluated -- check your --pairs and fundamentals
        return
    print(result.to_string(index=False))

    print("\n=== 3. Clearest Market Structure (top conviction, ranked) ===")
    top = result[result["Agreement"]].head(5)
    if top.empty:
        print("No pairs currently show fundamental/technical agreement.")
    else:
        for _, r in top.iterrows():
            direction = "bullish" if r["FundamentalBias"] > 0 else "bearish"
            print(f"  {r['Pair']}: {direction} {r['Base']} vs {r['Quote']} "
                  f"(conviction {r['ConvictionScore']:.2f}, structure={r['Structu


if __name__ == "__main__":
    main()
```